DR. RVR NRI INSTITUTE OF TECHNOLOGY

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

CONSOLIDATED REPORT — 10 SELECTED CASE STUDIES

Student Name	A. Balaji
Roll Number	26001374
Branch / Section	CSD-1 (Sec-1)
Course	Python Programming
Academic Year	2026–2027

Each case study includes a clear problem statement, objectives, assumptions, algorithm, proper symbol-based flowchart, pseudocode, Python source code, test cases, black-background output screenshot, and conclusion.

Question / Problem Statement

Calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates, then compare monthly bills and identify unusual increases.

Objectives

Compute a transparent bill from slab rates and compare consecutive monthly totals.

Assumptions

First 100 units cost Rs. 2/unit, next 100 cost Rs. 3/unit, and units above 200 cost Rs. 5/unit. Fixed charge is Rs. 50. A 5% surcharge applies when consumption exceeds 250 units; a Rs. 30 rebate applies when consumption is at most 100 units. An increase above 25% from the previous month is unusual.

Algorithm

1. Read monthly unit values and validate they are non-negative.
2. Calculate slab-wise energy charge, then fixed charge.
3. Apply surcharge or rebate using the stated rules.
4. Compare each month with the previous bill and flag increases above 25%.
5. Display the itemized bill and comparison.

Pseudocode

START

READ monthly_units

FOR each month:

IF units < 0: report invalid

ELSE calculate slab charge + fixed charge

IF units > 250: add 5% surcharge

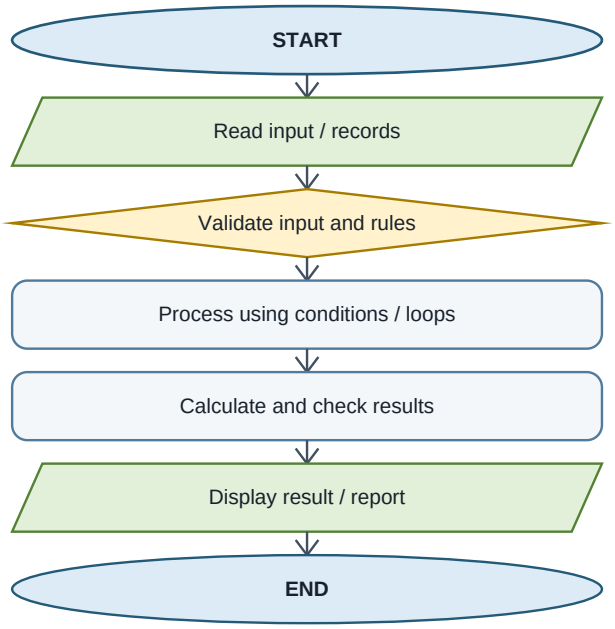
ELSE IF units <= 100: subtract Rs. 30 rebate, not below zero

compare bill with previous bill; flag increase > 25%

DISPLAY results

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 2 — Progressive Electricity Billing — Python Program and Testing

Python Source Code

```
def energy_charge(units):
    if units <= 100:
        return units * 2
    elif units <= 200:
        return 100 * 2 + (units - 100) * 3
    return 100 * 2 + 100 * 3 + (units - 200) * 5
monthly_units = [80, 120, 280]
previous_bill = None
for month, units in enumerate(monthly_units, start=1):
    if units < 0:
        print("Month", month, ": invalid units")
        continue
    energy = energy_charge(units)
    subtotal = energy + 50
    surcharge = subtotal * 0.05 if units > 250 else 0
    rebate = 30 if units <= 100 else 0
    bill = max(0, subtotal + surcharge - rebate)
    print(f"Month {month}: units={units}, bill=Rs.{bill:.2f}")
    if previous_bill is not None and previous_bill > 0:
        increase = (bill - previous_bill) / previous_bill * 100
        if increase > 25:
            print(f"  Unusual increase: {increase:.1f}%")
    previous_bill = bill
```

Test Cases

Input / Scenario

Expected check

Result

80 units

Energy Rs.160 + fixed Rs.50 – rebate Rs.30

Rs.180.00

120 units

Energy Rs.260 + fixed Rs.50

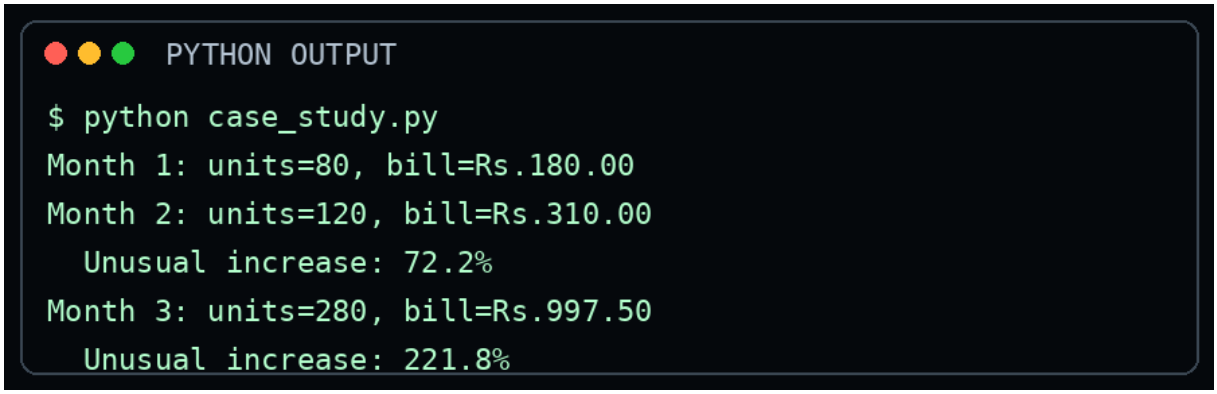
Rs.310.00

280 units

Energy Rs.900 + fixed Rs.50 + 5% surcharge

Rs.997.50

Sample Output Screenshot (black background)



Conclusion

The program applies progressive slabs, fixed charges, conditional surcharges/rebates, and flags unusually large month-to-month increases.

Question / Problem Statement

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Objectives

Calculate present attendance and the number of additional classes needed when every future class is attended.

Assumptions

Eligibility threshold is 75%. Future scenario assumes the student attends every additional class. Input counts must satisfy 0 <= attended <= total and total > 0.

Algorithm

1. Read attended and total classes.
2. Validate the counts.
3. Calculate current percentage.
4. Increase both attended and total by one until attendance reaches 75%.
5. Display percentage and extra classes required.

Pseudocode

START

READ attended, total

IF total <= 0 or attended < 0 or attended > total: print invalid

ELSE:

percentage = attended / total * 100

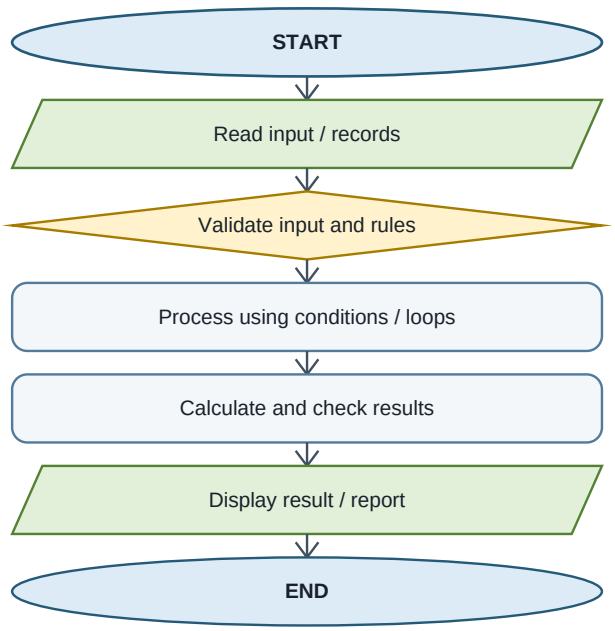
extra = 0

WHILE (attended + extra) / (total + extra) < 0.75: extra = extra + 1

DISPLAY percentage and extra

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 12 — Attendance Recovery Computation — Python Program and Testing

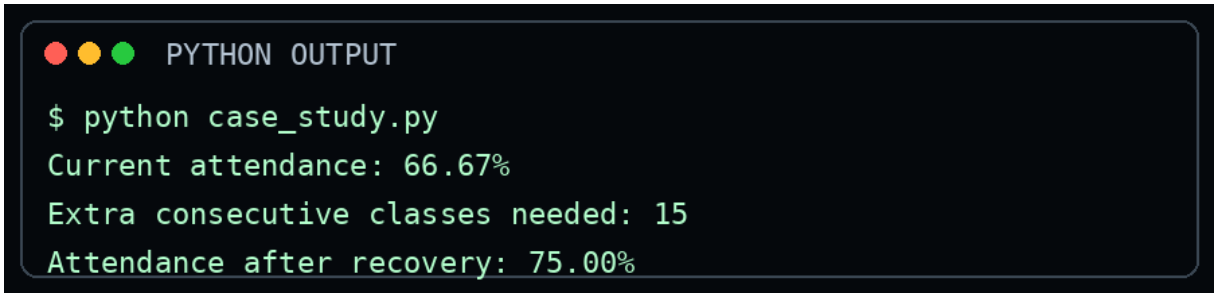
Python Source Code

```
attended = 30
total = 45
if total <= 0 or attended < 0 or attended > total:
    print("Invalid attendance data")
else:
    percentage = attended / total * 100
    extra = 0
    while (attended + extra) / (total + extra) < 0.75:
        extra += 1
    print(f"Current attendance: {percentage:.2f}%")
    print("Extra consecutive classes needed:", extra)
    print(f"Attendance after recovery: {(attended + extra) / (total + extra) * 100:.2f}%")
```

Test Cases

Input / Scenario	Expected check	Result
30 attended / 45 total	66.67% current	15 extra classes; final 75.00%
38 / 50	76.00% current	0 extra classes
5 / 0	invalid total	Invalid attendance data

Sample Output Screenshot (black background)



Conclusion

The loop finds the minimum number of fully attended future classes needed to reach the attendance threshold.

Question / Problem Statement

Analyse hypothetical chemical inventory records using storage categories, maximum quantities, and compatibility restrictions; flag invalid combinations.

Objectives

Validate records against storage limits and a supplied incompatibility rule without handling real chemicals.

Assumptions

This is a classroom-only hypothetical dataset. Maximum quantity is 20 units per storage category. Records in incompatible categories ACID and BASE must not share a storage cabinet. This program is not a real chemical-safety guide.

Algorithm

1. Load sample records as tuples: name, category, quantity, cabinet.
2. Validate non-negative quantities and category limits.
3. Group records by cabinet.
4. Flag cabinets containing both ACID and BASE categories.
5. Print each finding.

Pseudocode

START

LOAD inventory tuples

FOR each record: validate quantity and category limit

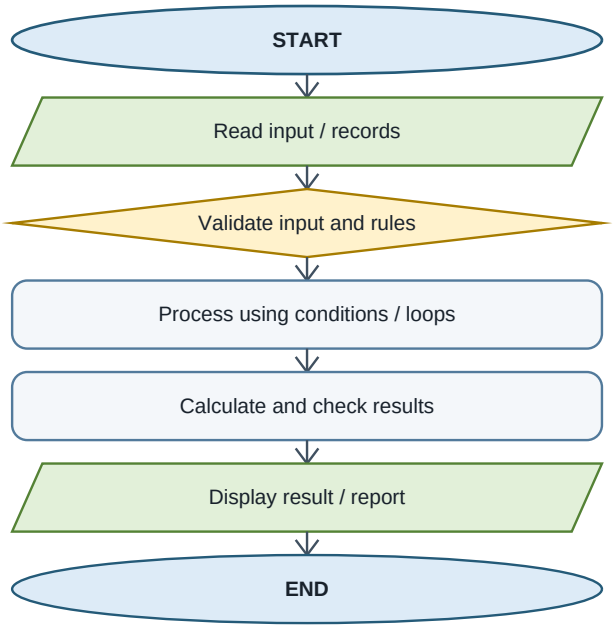
GROUP records by cabinet

FOR each cabinet: IF ACID and BASE both present, flag conflict

DISPLAY validation report

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 22 — Laboratory Chemical Inventory Validation — Python Program and Testing

Python Source Code

```
inventory = [
    ("Sample A", "ACID", 8, "Cabinet-1"),
    ("Sample B", "BASE", 5, "Cabinet-1"),
    ("Sample C", "NEUTRAL", 12, "Cabinet-2"),
    ("Sample D", "ACID", 24, "Cabinet-3")
]
max_quantity = 20
cabinet_categories = {}
issues = []
for name, category, quantity, cabinet in inventory:
    if quantity < 0:
        issues.append(f"{name}: negative quantity")
    if quantity > max_quantity:
        issues.append(f"{name}: quantity exceeds {max_quantity}")
    cabinet_categories.setdefault(cabinet, set()).add(category)
for cabinet, categories in cabinet_categories.items():
    if "ACID" in categories and "BASE" in categories:
        issues.append(f"{cabinet}: incompatible category combination")
if issues:
    for issue in issues:
        print("FLAG:", issue)
else:
    print("All sample records passed validation")
```

Test Cases

Input / Scenario

Expected check

Result

Sample A and B in

Cabinet-1

ACID + BASE

Cabinet conflict flagged

Sample D quantity 24

limit 20

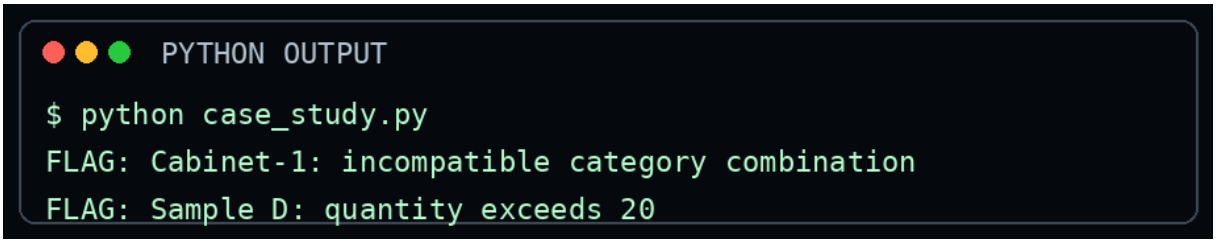
Quantity limit flagged

Sample C

12 units, NEUTRAL

No quantity issue

Sample Output Screenshot (black background)



Conclusion

Tuple records and grouped cabinet categories make it possible to flag sample data that violates the explicitly supplied classroom rules.

Question / Problem Statement

Analyse multiple feedback sentences using a supplied keyword list, count positive and negative indicators, calculate scores, and classify feedback using predefined thresholds.

Objectives

Use basic string and list operations to provide a simple, explainable sentiment score.

Assumptions

Positive keywords are good, clear, helpful, excellent. Negative keywords are poor, confusing, slow, difficult. Score = positive hits – negative hits. Scores above zero are Positive, below zero Negative, and zero Neutral. This is a simple keyword exercise, not a reliable opinion classifier.

Algorithm

1. Store feedback sentences and keyword lists.
2. Convert each sentence to lowercase and split into words.
3. Count positive and negative keyword matches.
4. Subtract negative count from positive count.
5. Classify and print each sentence's score.

Pseudocode

START

SET positive_words and negative_words

FOR each feedback sentence:

words = lowercase sentence split into words

count positive and negative keyword matches

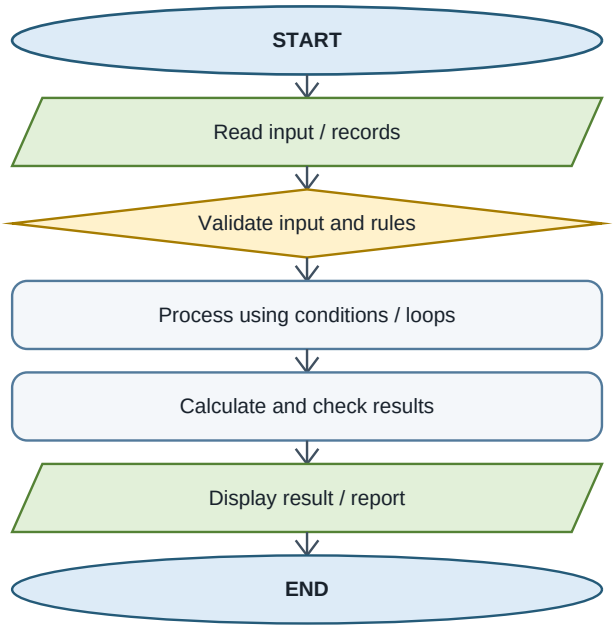
score = positive - negative

classify score as Positive, Negative, or Neutral

DISPLAY counts and class

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 34 — Student Feedback Sentiment Scoring — Python Program and Testing

Python Source Code

```
feedback = [
    "The lessons are clear and helpful",
    "The lab was slow and confusing",
    "The course is good but difficult"
]
positive_words = ["good", "clear", "helpful", "excellent"]
negative_words = ["poor", "confusing", "slow", "difficult"]
for sentence in feedback:
    words = sentence.lower().replace(".", "").split()
    positive = sum(1 for word in words if word in positive_words)
    negative = sum(1 for word in words if word in negative_words)
    score = positive - negative
    if score > 0:
        label = "Positive"
    elif score < 0:
        label = "Negative"
    else:
        label = "Neutral"
    print(f"{sentence} | +{positive} -{negative} | score={score} | {label}")
```

Test Cases

Input / Scenario

Expected check

Result

clear and helpful

2 positive, 0 negative

Positive, score +2

slow and confusing

0 positive, 2 negative

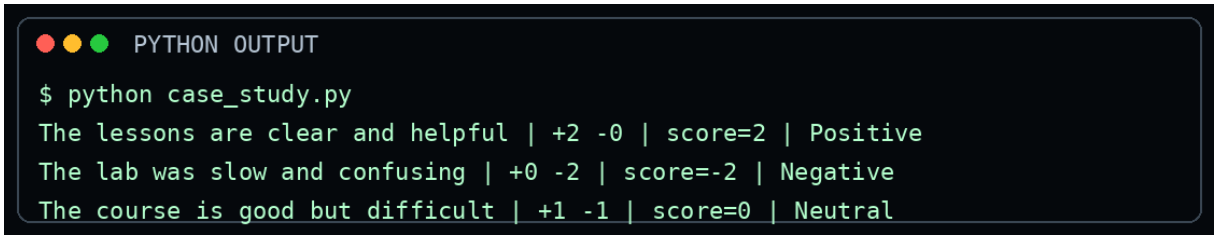
Negative, score -2

good but difficult

1 positive, 1 negative

Neutral, score 0

Sample Output Screenshot (black background)



Conclusion

The program demonstrates keyword matching, counting, arithmetic scoring, and conditional classification.

Question / Problem Statement

Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Objectives

Use nested lists and loops to calculate student totals, subject averages, top performer, and low-average subjects.

Assumptions

Rows represent students and columns represent Mathematics, Python, and English. Marks are from 0 to 100. A subject average below 50 is considered low.

Algorithm

1. Store marks in a nested list and names in a list.
2. Calculate each student's average.
3. Calculate each subject's average by column.
4. Identify the highest student average.
5. Flag subjects averaging below 50.

Pseudocode

START

LOAD names, subjects, marks matrix

FOR each student row: calculate average

FOR each subject column: calculate average

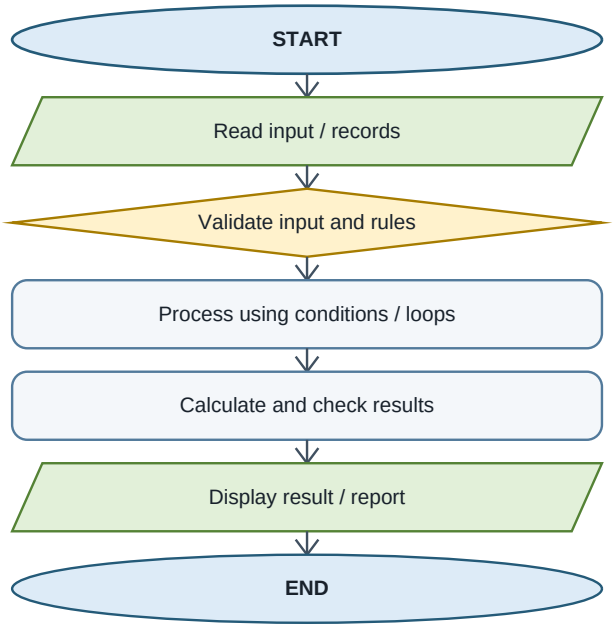
FIND student with highest average

FLAG subject average < 50

DISPLAY all summaries

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 42 — Student Marks Matrix Analysis — Python Program and Testing

Python Source Code

```
names = ["Asha", "Bharat", "Charan", "Divya"]
subjects = ["Maths", "Python", "English"]
marks = [
    [78, 88, 74],
    [45, 52, 48],
    [91, 95, 89],
    [62, 70, 66]
]
averages = []
for name, row in zip(names, marks):
    average = sum(row) / len(row)
    averages.append(average)
    print(f"{name} average: {average:.2f}")
subject_averages = []
for column in range(len(subjects)):
    average = sum(row[column] for row in marks) / len(marks)
    subject_averages.append(average)
    print(f"{subjects[column]} average: {average:.2f}")
top_index = averages.index(max(averages))
print("Top performer:", names[top_index])
for subject, average in zip(subjects, subject_averages):
    if average < 50:
        print("Low-average subject:", subject)
```

Test Cases

Input / Scenario

Expected check

Result

Asha: 78,88,74

average 80.00

Row average correct

Python marks 88,52,95,70

column average 76.25

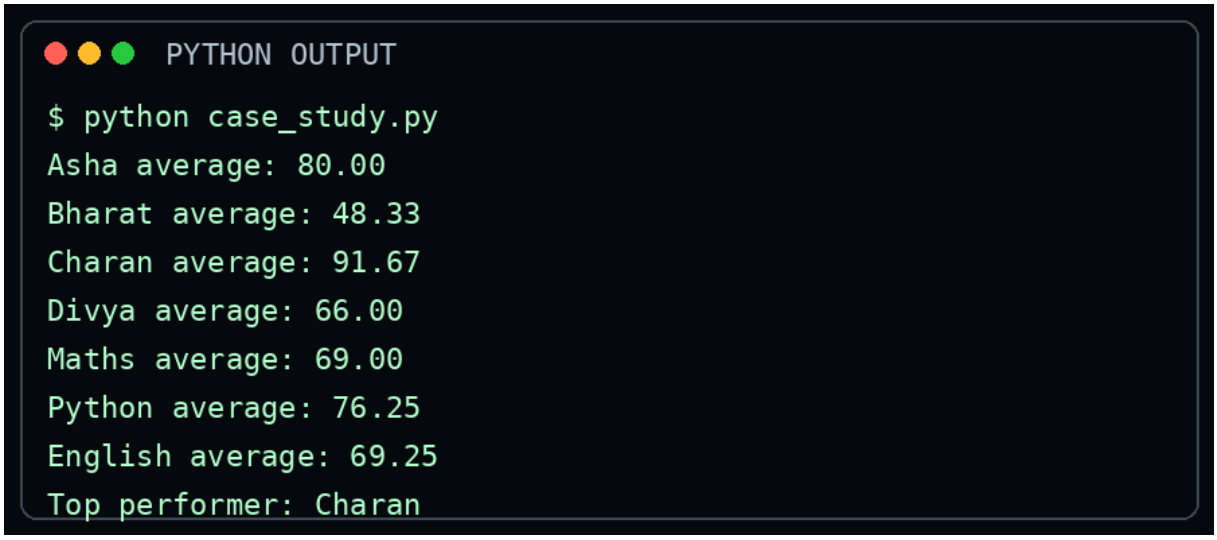
Subject average correct

Divya vs others

Divya average 66.00; Charan 91.67

Charan is top performer

Sample Output Screenshot (black background)



Conclusion

Nested-list processing supports row-wise and column-wise summaries and identifies the highest-scoring student.

Question / Problem Statement

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

Objectives

Track passenger occupancy from stop to stop and ensure counts stay within bus capacity.

Assumptions

The bus starts empty and has capacity 40. At each stop, alighting passengers leave before new passengers board. Counts must be non-negative and alighting cannot exceed the current passenger count.

Algorithm

1. Set capacity and initial passenger count.
2. Read each stop's boarding and alighting counts.
3. Validate counts and alighting availability.
4. Update passenger count after alighting and boarding.
5. Flag occupancy above capacity and display the stop summary.

Pseudocode

START

```
capacity = 40; passengers = 0
```

FOR each stop tuple (stop, board, alight):

validate non-negative counts and alight <= passengers

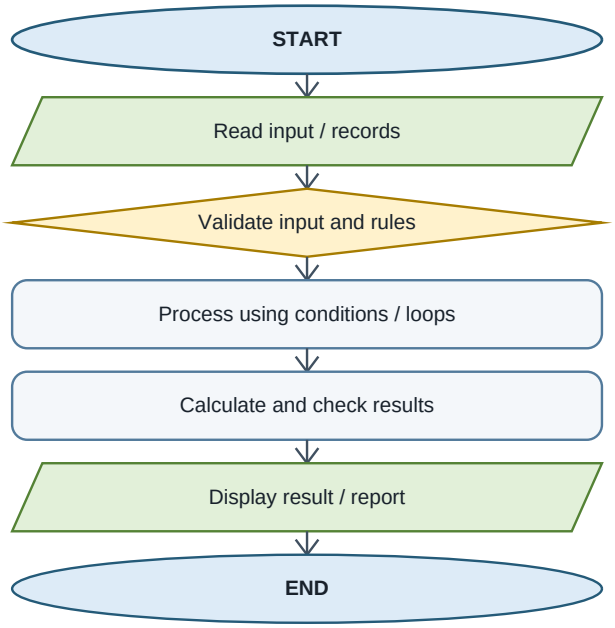
passengers = passengers - alight + board

IF passengers > capacity: flag capacity violation

DISPLAY passenger count

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 51 — Bus Passenger Occupancy Simulation — Python Program and Testing

Python Source Code

```
capacity = 40
passengers = 0
stops = [
    ("Central", 18, 0),
    ("Market", 12, 5),
    ("College", 15, 8),
    ("Depot", 10, 4)
]
for stop, boarding, alighting in stops:
    if boarding < 0 or alighting < 0 or alighting > passengers:
        print(f"{stop}: invalid passenger counts")
        continue
    passengers = passengers - alighting + boarding
    status = "CAPACITY EXCEEDED" if passengers > capacity else "OK"
    print(f"{stop}: passengers={passengers}/{capacity} [{status}]")
```

Test Cases

Input / Scenario

Expected check

Result

Central: board 18, alight 0

18 passengers

OK

College after previous stop

32 passengers

OK

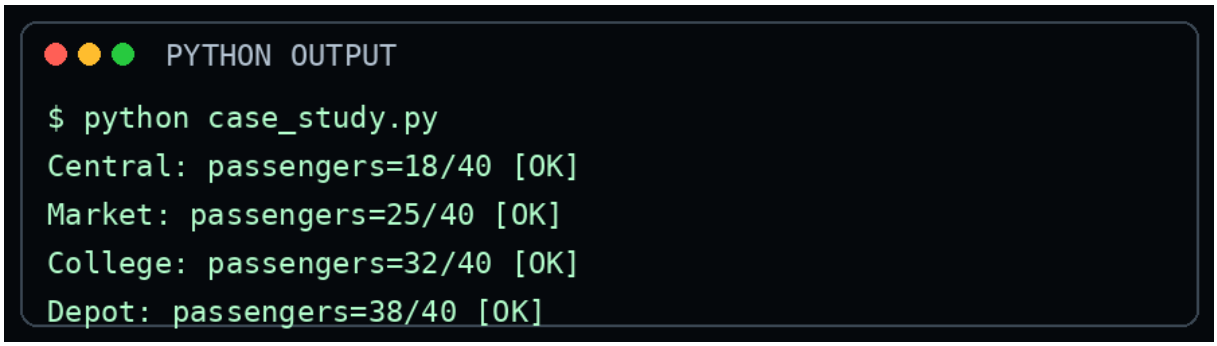
Test board 25 while 32

aboard

57 passengers

capacity exceeded

Sample Output Screenshot (black background)



Conclusion

The simulation updates occupancy at each stop and checks the resulting count against the bus capacity.

Question / Problem Statement

Represent course registrations as tuples, identify duplicate registrations and invalid course combinations, and determine students whose selections violate supplied rules.

Objectives

Validate tuple-based registrations using duplicate checks and course-combination constraints.

Assumptions

Each registration tuple contains (student_id, course_code). A student may register for a course only once. LAB201 requires PY101. This is a hypothetical rule set for demonstrating tuples.

Algorithm

1. Load registrations as tuples.
2. Track seen student-course pairs and flag duplicates.
3. Build a set of courses for each student.
4. Flag students registered in LAB201 without PY101.
5. Display all validation issues.

Pseudocode

START

LOAD registration tuples

FOR each tuple: flag duplicate pair if already seen

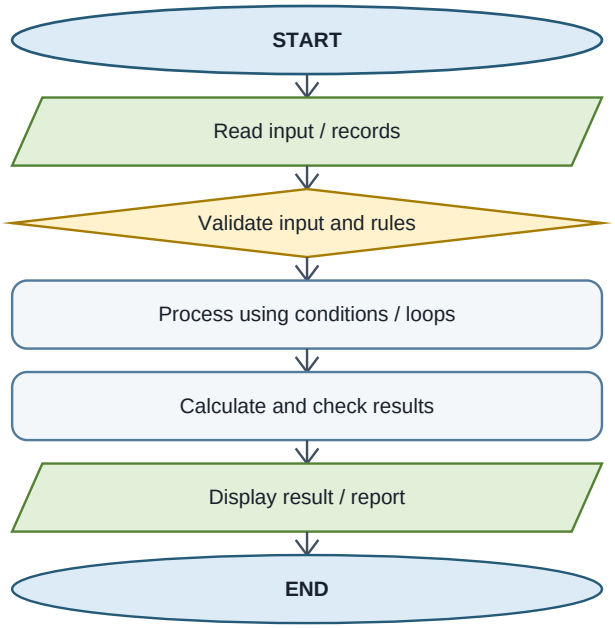
GROUP course codes by student

FOR each student: IF LAB201 present and PY101 absent, flag

DISPLAY issues

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 64 — Tuple-Based Course Registration Validation — Python Program and Testing

Python Source Code

```
registrations = [
    ("S101", "PY101"),
    ("S101", "LAB201"),
    ("S102", "LAB201"),
    ("S103", "MA101"),
    ("S101", "PY101")
]
seen = set()
student_courses = {}
issues = []
for student, course in registrations:
    pair = (student, course)
    if pair in seen:
        issues.append(f"Duplicate registration: {student} - {course}")
    seen.add(pair)
    student_courses.setdefault(student, set()).add(course)
for student, courses in student_courses.items():
    if "LAB201" in courses and "PY101" not in courses:
        issues.append(f"{student}: LAB201 requires PY101")
if issues:
    for issue in issues:
        print("FLAG:", issue)
else:
    print("All registrations are valid")
```

Test Cases

Input / Scenario

Expected check

Result

S101, PY101 appears

twice

duplicate pair

Duplicate flagged

S102 has LAB201 only

prerequisite missing

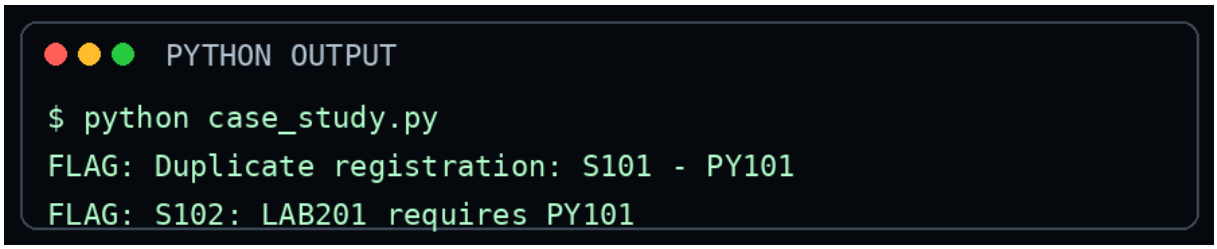
Rule violation flagged

S103 has MA101

no conflict

No issue

Sample Output Screenshot (black background)



Conclusion

Tuple records, sets, and grouped course selections can detect duplicate registrations and invalid course combinations.

Question / Problem Statement

Process order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

Objectives

Reconcile ordered, dispatched, and returned quantities to find pending units and net accepted deliveries.

Assumptions

Each record is (order_id, ordered, dispatched, returned). All values are non-negative; dispatched cannot exceed ordered; returned cannot exceed dispatched. Outstanding = ordered – dispatched. Net accepted = dispatched – returned.

Algorithm

1. Read order records as tuples.
2. Validate all quantities and relationships.
3. Calculate outstanding and net accepted quantities.
4. Update summary totals for valid records.
5. Flag inconsistent records.

Pseudocode

START

LOAD order tuples

FOR each order:

IF values invalid or dispatched > ordered or returned > dispatched: flag

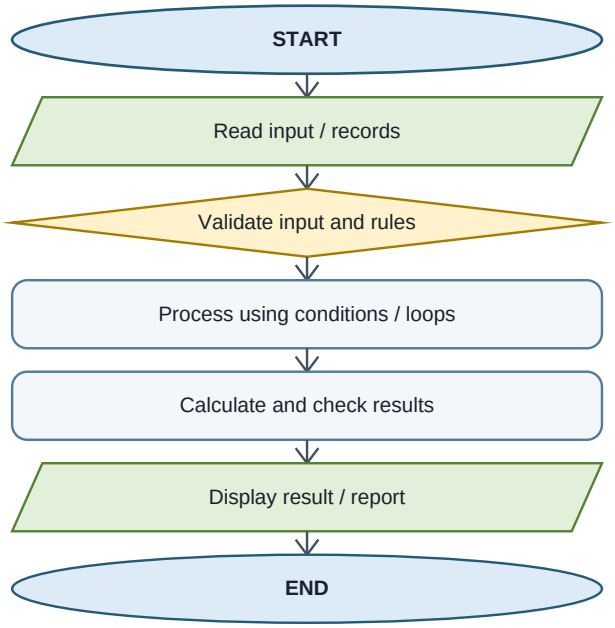
ELSE calculate outstanding and net accepted

update totals

DISPLAY per-order and totals

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 67 — Retail Order Dispatch and Return Analysis — Python Program and Testing

Python Source Code

```
orders = [
    ("ORD-101", 50, 40, 3),
    ("ORD-102", 30, 30, 2),
    ("ORD-103", 20, 25, 0),
    ("ORD-104", 15, 10, 12)
]
total_outstanding = 0
total_accepted = 0
for order_id, ordered, dispatched, returned in orders:
    if min(ordered, dispatched, returned) < 0 or dispatched > ordered or returned > dispatched:
        print(f"{order_id}: INCONSISTENT RECORD")
        continue
    outstanding = ordered - dispatched
    accepted = dispatched - returned
    total_outstanding += outstanding
    total_accepted += accepted
    print(f"{order_id}: outstanding={outstanding}, net accepted={accepted}")
print("Total outstanding:", total_outstanding)
print("Total net accepted:", total_accepted)
```

Test Cases

Input / Scenario

Expected check

Result

ORD-101: 50,40,3

10 outstanding; 37 accepted

Valid

ORD-103: 20,25,0

dispatch exceeds order

Inconsistent

ORD-104: 15,10,12

returns exceed dispatch

Inconsistent

Sample Output Screenshot (black background)



Conclusion

The reconciliation distinguishes pending orders from accepted deliveries and prevents inconsistent records from affecting totals.

Question / Problem Statement

Process deposits, withdrawals, and transfers in a simulated account, maintain balances, detect invalid operations, and report suspicious repeated patterns.

Objectives

Simulate a sequence of account transactions and identify invalid operations and repeated withdrawals.

Assumptions

This is a fictional classroom simulation, not connected to a bank. Opening balance is Rs. 5,000. Deposits must be positive. Withdrawals and transfers must be positive and cannot exceed available balance. Two or more consecutive withdrawals of Rs. 2,000 or more are flagged for review.

Algorithm

1. Initialize balance and transaction history.
2. Process each transaction tuple.
3. Validate amount and sufficient funds for withdrawals/transfers.
4. Update balance for valid transactions and retain transaction history.
5. Flag consecutive high-value withdrawals and print the audit summary.

Pseudocode

START

balance = 5000

FOR each (type, amount): validate amount

DEPOSIT: add amount

WITHDRAW/TRANSFER: require amount <= balance, then subtract

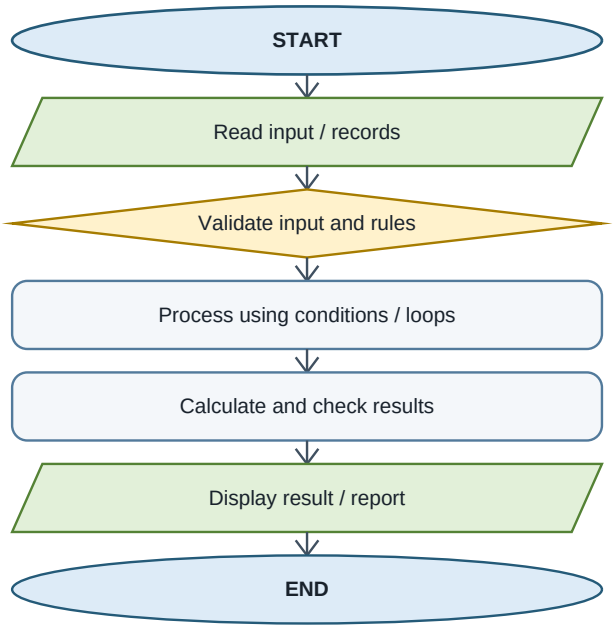
record valid transaction

track consecutive withdrawals >= 2000

DISPLAY balance, invalid operations, review flags

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 86 — Multi-Stage ATM Transaction Audit — Python Program and Testing

Python Source Code

```
transactions = [
    ("DEPOSIT", 1000),
    ("WITHDRAW", 2200),
    ("WITHDRAW", 2500),
    ("TRANSFER", 1800),
    ("WITHDRAW", 9000)
]
balance = 5000
high_withdrawals = 0
for kind, amount in transactions:
    if amount <= 0:
        print("INVALID:", kind, amount)
        continue
    if kind == "DEPOSIT":
        balance += amount
        high_withdrawals = 0
    elif kind in ("WITHDRAW", "TRANSFER"):
        if amount > balance:
            print("REJECTED: insufficient funds for", kind, amount)
            high_withdrawals = 0
            continue
        balance -= amount
        if kind == "WITHDRAW" and amount >= 2000:
            high_withdrawals += 1
            if high_withdrawals >= 2:
                print("REVIEW: repeated high-value withdrawals")
        else:
            high_withdrawals = 0
    else:
        print("INVALID transaction type:", kind)
    print(f"Balance after {kind}: Rs.{balance:.2f}")
print(f"Final balance: Rs.{balance:.2f}")
```

Test Cases

Input / Scenario

Expected check

Result

Deposit Rs.1000

balance 6000

Accepted

Withdraw Rs.2200 then

Rs.2500

balance 1300

Repeated high-value withdrawal

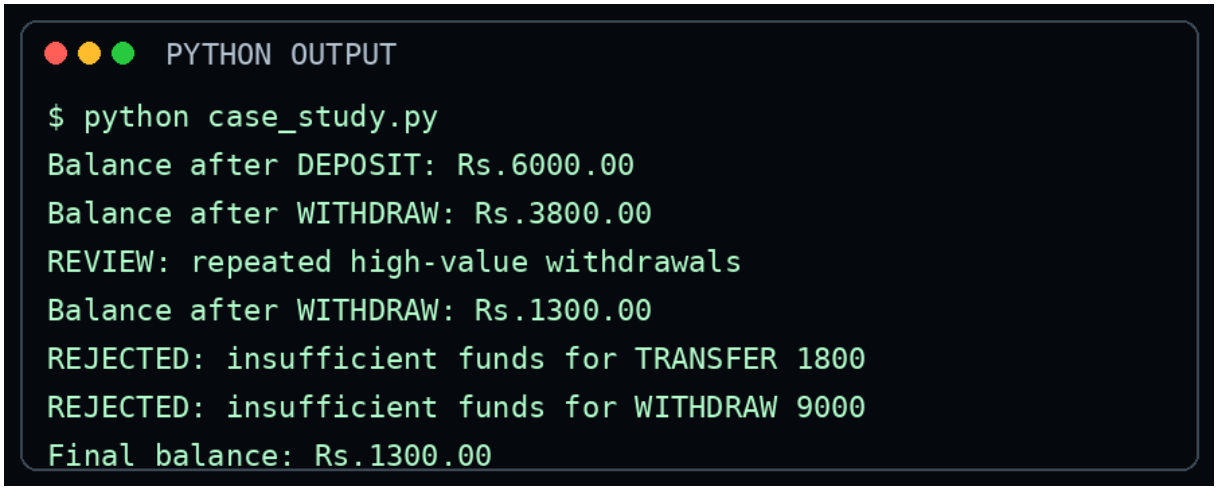
flagged

Withdraw Rs.9000

amount exceeds balance

Rejected

Sample Output Screenshot (black background)



Conclusion

The audit updates a simulated balance, rejects unaffordable transactions, and flags a simple repeated-withdrawal pattern for review.

Question / Problem Statement

Represent event schedules and team scores using lists and tuples, identify scheduling conflicts, aggregate points, and generate final rankings.

Objectives

Detect overlapping events assigned to the same venue and rank teams by total points.

Assumptions

Event tuple format is (event, venue, start_hour, end_hour). Intervals overlap if max(start1,start2) < min(end1,end2). Team score tuples are (team, points). Higher total points rank higher; ties share the same points total and are ordered alphabetically for display.

Algorithm

1. Load event schedules and team scores.
2. Validate event intervals.
3. Compare each pair of events at the same venue for overlap.
4. Aggregate points by team.
5. Sort teams by descending total points and display rankings.

Pseudocode

START

LOAD event tuples and score tuples

COMPARE event pairs: same venue and overlapping interval -> flag

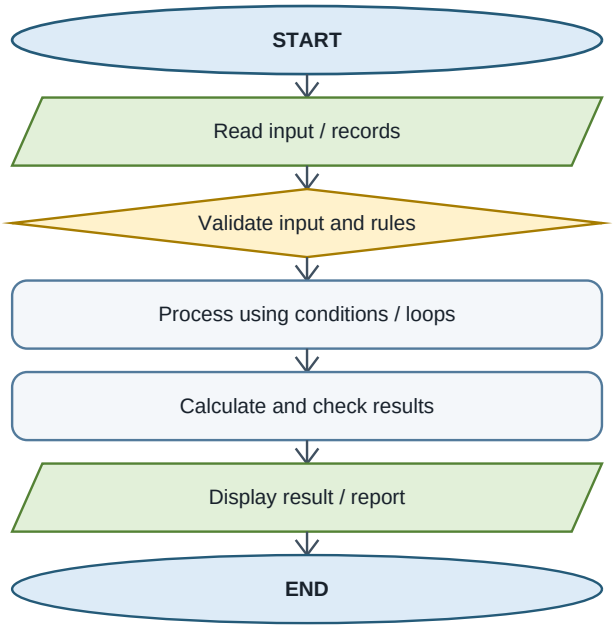
SUM points for each team

SORT by points descending, then name

DISPLAY conflicts and rankings

END

Flowchart



Flowchart: standard symbols and directional flow

CASE STUDY 92 — Multi-Event Sports Scheduling and Scoring — Python Program and Testing

Python Source Code

```
events = [
    ("100m Final", "Track", 10, 11),
    ("Long Jump", "Field", 10, 12),
    ("Relay", "Track", 10.5, 11.5),
    ("Shot Put", "Field", 13, 14)
]
scores = [
    ("Falcons", 12), ("Tigers", 18), ("Falcons", 7),
    ("Eagles", 18), ("Tigers", 4), ("Eagles", 6)
]
for i in range(len(events)):
    for j in range(i + 1, len(events)):
        a, b = events[i], events[j]
        if a[1] == b[1] and max(a[2], b[2]) < min(a[3], b[3]):
            print(f"CONFLICT: {a[0]} overlaps {b[0]} at {a[1]}")
totals = {}
for team, points in scores:
    totals[team] = totals.get(team, 0) + points
ranking = sorted(totals.items(), key=lambda item: (-item[1], item[0]))
print("Final rankings:")
for position, (team, points) in enumerate(ranking, start=1):
    print(f"{position}. {team}: {points} points")
```

Test Cases

Input / Scenario

Expected check

Result

100m Final and Relay

same Track, 10–11 and 10.5–11.5

Conflict flagged

Tigers scores 18 + 4

22 points

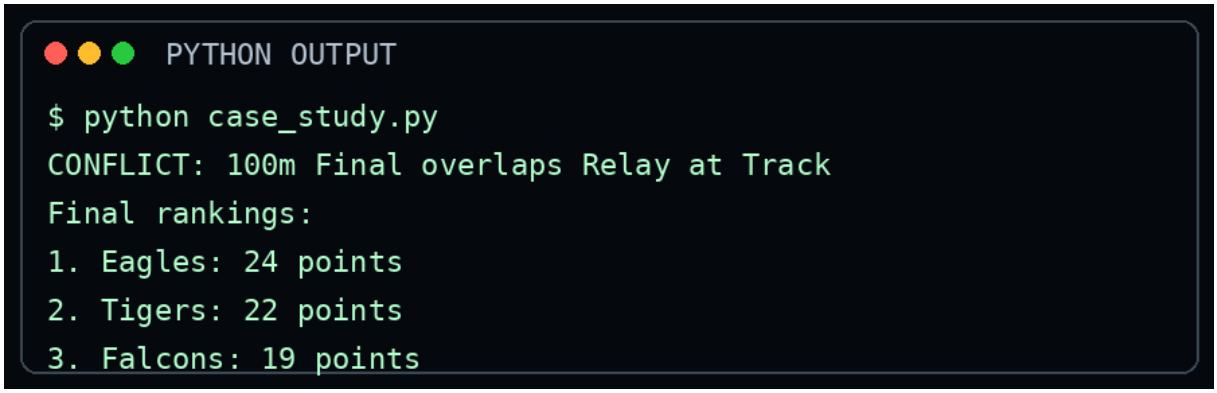
Top rank

Eagles scores 18 + 6

24 points

Top rank; Eagles ahead of Tigers

Sample Output Screenshot (black background)



Conclusion

The program combines tuple-based schedule comparison with score aggregation and deterministic ranking.